

Day2 · 12차시

# 멀티에이전트 & 오케스트레이션

한 명의 천재보다, 잘 조율된 팀이 멀리 간다.  
한 에이전트를 여럿으로 나누고 조율하고, 그 협업을 하네스로 떠받친다.

SECTION 00

# 오프닝 — 하나에서 여럿으로

하나로 안 되면 더 똑똑한 모델? 아니다 — 역할을 나누고, 그 협업을 떠받칠 무대를 짠다.

**“하나로 안 되면 더 똑똑한 모델? 아니다 — 역할을 나누고, 그 협업을 떠받칠  
하네스를 짠다.”**

— 11차시(한 에이전트 똑똑하게) → 12차시(여럿으로 조율)

## 지금까지 vs 오늘

▶ 한 에이전트를 똑똑하게 → 여럿을 나누고 조율하게 (저녁 팀 MVP 워밍업)

### 지금까지 (7~11차시)

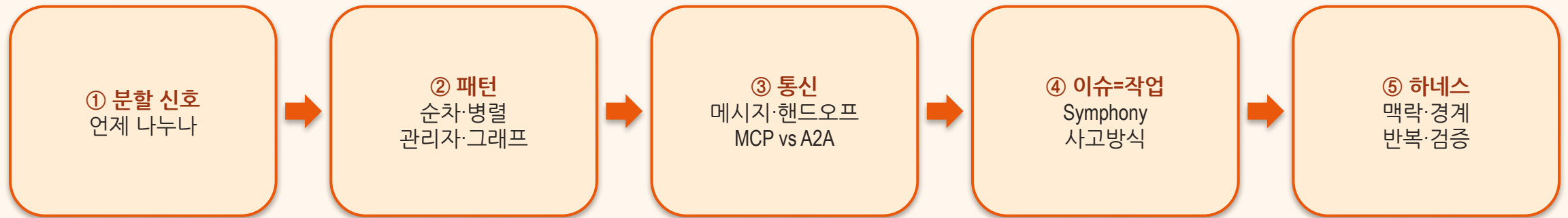
- 추론 · 도구 · 메모리 · 지식(RAG)
- **한 에이전트**를 똑똑하게
- 혼자서 모든 일을 처리

VS

### 오늘 (12차시)

- 그 에이전트를 **여럿으로 나누고 조율**
- 협업을 떠받치는 하네스 (맥락·경계·반복·검증)
- 팀 MVP → 계획자+실행자 팀으로

## 이 차시 동안 얻는 것



SECTION 01

# 언제 에이전트를 나누나

## 혼자 저글링 vs 분담

▶ 연구 보조 예시 — 사람 조직과 똑같다

### 한 에이전트가 전부

- 검색·가공·비평·요약을 혼자 저글링
- 컨텍스트가 엉킨다
- 길이·비용이 폭발
- 큰 용례로 확장 한계

VS

### 전문 에이전트로 분담

- 검색·처리·논문·요약 에이전트 분리
- 각자 자기 전문 영역만 집중
- 병렬로 더 빠르게
- 역할마다 다른 도구·모델 가능

# 이럴 때 나눠라

▶ 이득이 분명할 때만 나눈다 — 네 가지 신호

1

## 독립 하위작업

작업이 독립 하위작업으로 깔끔히 쪼개진다

2

## 다른 전문성 · 도구

작업마다 필요한 전문성/도구가 서로 다르다

3

## 병렬 이득

동시에 돌리면 확실히 빨라진다

4

## 컨텍스트 과부하

한 프롬프트에 다 넣기엔 컨텍스트가 너무 크다



## 세 가지 에이전트 구성

▶ 우리 실습은 이질(Planner ≠ Executor) 구성에 해당

### ① 동질 (Homogeneous)

같은 능력 → 병렬로  
효율 처리 — 같은 일을  
여러 일꾼이 분담

### ② 이질 (Heterogeneous)

역할·도구·모델·메모리가  
서로 다른 전문가들

### ③ 창발적 전문화

처음엔 같다가 상호작용  
속에서 역할이 스스로 분화

## 공짜 점심이 아니다

▶ ‘나누면 무조건 좋다’는 착각을 경계 — 분할 신호가 분명할 때만

### 이득

- 협력: 복잡한 문제를 함께
- 확장성: 병렬·비동기로 수를 늘려도 OK
- 속도: 전문 에이전트가 각자 더 빠르게

VS

### 대가

- 복잡성: 오케스트레이션·추상화 계층↑
- 비용: 강력한 LLM × 에이전트 수
- 평가: 여럿이 얹히면 디버깅·평가 난이도↑

## 여럿이라서 생기는 사고 3가지

### 오류 연쇄

한 에이전트의 오류가  
다음 입력을 오염 —  
순차일수록 지수적 증폭  
→ 검증 게이트로 조기 차단

### 비용 폭발

$N$  에이전트  $\times$  비싼 LLM  
= 단일 대비 10~100x  
→ 실행은 싼 모델,  
계획만 프리미엄

### 에이전트 난립

수가 늘면 통신량은  
제공으로 증가  
→ 팬아웃 제한 +  
계층으로 묶기

SECTION 02

# 누가 누구를 조율하나

## 오케스트레이션이란

▶ 단순 작업엔 필요 없다 — 여럿이 얹힌 작업일수록 조율이 본체

무엇

도구·모델·데이터를 엮어 **질의**를 **결과**로 바꾸는 핵심 로직

질문 ①

“어떤 도구를 **언제**” 부를 것인가

질문 ②

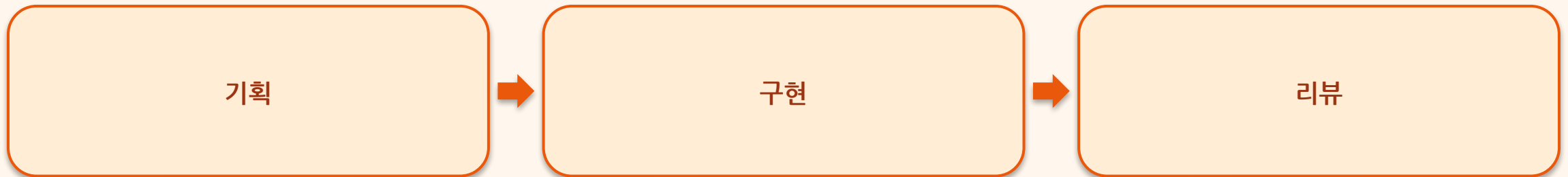
각 호출에 **올바른 컨텍스트**를 어떻게 넣을 것인가

패턴이란

에이전트들의 위상 구조 — 누가 누구와 통신하나

## 순차 (Sequential) — 파이프라인

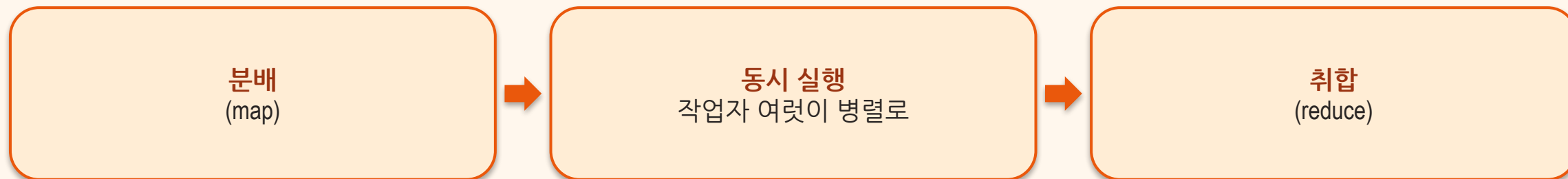
▶ 산출물(명세·코드)이 다음 단계의 입력 — MetaGPT의 SOP(Standard Operating Procedure) 방식



🔄 한 단계가 틀리면 오류가 뒤로 그대로 전파 — 최대 길이를 제한하라

## 병렬 (Parallel) — map → reduce

▶ 독립 하위작업을 동시에 → 결과 합치기 — 기다림은 가장 느린 작업만큼만



## 관리자-작업자 (Manager-Worker)

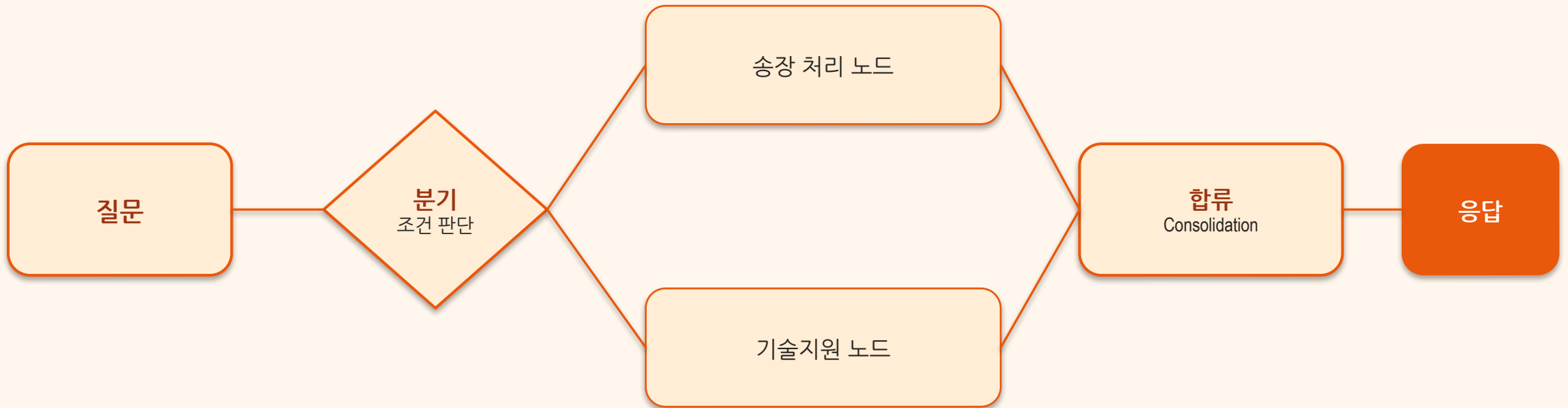
▶ 가장 흔하고 구현이 쉽다 — 단, 관리자가 죽으면 전체 정지(단일 실패 지점)





## 그래프 (Graph) — 조건 분기와 합류

▶ 비선형 흐름 — 분기·합류·조건 라우팅. 실행 전에 사이클·도달불가 노드를 검증하라

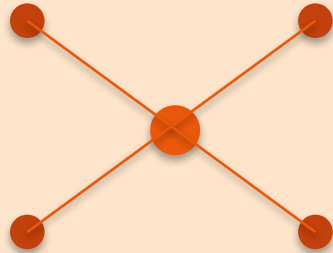


## 누가 통신을 책임지나

▶ 정답은 없다 — 쉬우면 중앙집중, 견고하면 분산, 효율 분담은 계층, 조직 경계는 연합

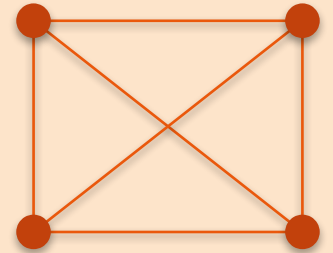
### 중앙집중 (Centralized)

허브가 전부 할당 — 쉬움  
단일 실패 지점



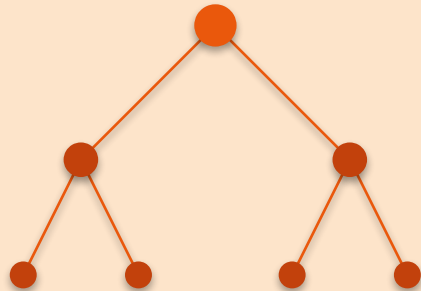
### 분산 (Decentralized)

리더 없는 수평 협력  
장애에 강함·통신량↑



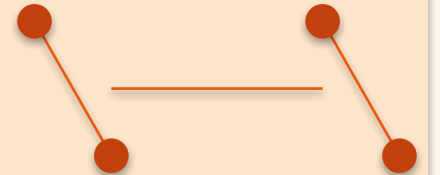
### 계층 (Hierarchical)

층마다 역할·권한  
효율적 분담



### 연합 (Federated)

조직 경계 넘어 간접 통신  
프라이버시·표준 필요



## 패턴(통신) ↔ 아키타입(역할): Planner-Executor

▶ ReAct는 매 스텝 비싼 호출 — P-E는 비싼 계획 1번 + 실행 N번

### 패턴 = 통신 구조

- 에이전트들의 위상
- 순차 · 병렬 · 관리자 · 그래프
- 누가 누구와 대화하나

VS

### 아키타입 = 역할 구조

- **Planner**(큰 모델): 다단계 계획 1번
- **Executor**(작은 모델/도구): 실행 N번
- 계획·실행 분리 → 디버깅·비용↓

## 상태(state)가 자라며 흐른다

▶ 각 노드는 읽고 '추가'만 한다 — 말없이 덮어쓰지 않는다 (append, no clobber)

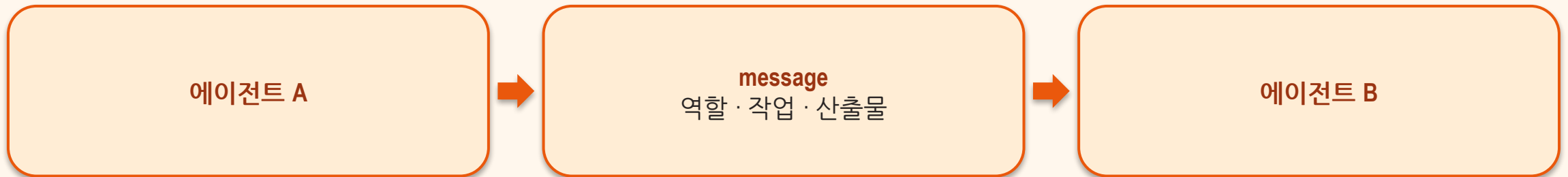


SECTION 03

# 에이전트는 어떻게 대화하나

## 무엇을 주고받나

▶ 에이전트 간 통신 = 컨텍스트 엔지니어링의 연장 — 많아도·적어도·형식이 틀려도 성능이 깎인다



☞ 자연어 수다보다 구조화된 산출물(명세·플랜·JSON)이 안정적 — MetaGPT 교훈

## 일을 넘긴다 — 4단계

▶ 작업만 던지는 게 아니라 맥락까지 같이 넘긴다

1

### 발견 (Discovery)

누가 뭘 할 수 있나 (능력 파악)

2

### 위임 (Delegate)

제어권 + 작업 + 맥락을 넘김

3

### 실행 (Execute)

받은 에이전트가 도구로 처리

4

### 산출물 반환

결과 + artifact를 돌려준다

## MCP vs A2A — 무엇을 표준화하나

▶ 둘은 보완적 — MCP로 도구를, A2A로 동료 에이전트를 잇는다

### MCP

- 에이전트 ↔ 도구
- 외부 도구·데이터 접근 표준
- 9차시에서 다룸

VS

### A2A (Agent2Agent)

- 에이전트 ↔ 에이전트
- 발견·인증·스트리밍·다중턴
- 에이전트 카드로 능력 노출



SECTION 04

## 트래커를 상태 머신으로 (Symphony)

## 감독을 트래커에 위임

▶ 한 사람이 감독 가능한 동시 세션은 3~5개 — 그 이상은 트래커가 감독한다

1

### TODO / REWORK

열린 작업 — 에이전트가 가져감

2

### IN PROGRESS

전용 워크스페이스에서 에이전트가 구현·테스트

3

### HUMAN REVIEW

작업 증명 첨부 → 사람은 검토만

4

### MERGING

충돌 해결·CI 재시도 → 머지

## 이슈가 곧 작업 단위

▶ 캠프 수준: 도입까진 아니어도 '이슈=작업' 사고방식이 핵심

### 원칙

“PR·세션은 수단, 목적은 이슈/작업” — day1 협업 구조의 일반화

### 매핑

열린 이슈 1개 = 에이전트 워크스페이스 1개

### 병렬

막히지 않은 작업부터 병렬 실행

### 분리

범위 밖 개선은 새 이슈로 — 이슈는 틀려도 비용 0

SECTION 05 · 핵심

# 하네스 엔지니어링

모델을 감싸 추론을 실제 작업으로 바꾸는 모든 것 — 그리고 왜 멀티에이전트엔 필수인가

## 하네스란 무엇인가

▶ ‘좋은 결과’를 얻는다 = 하네스를 내 프로젝트에 맞게 세팅하는 일 — 모델 교체가 아니라 하네스 튜닝

사람의 목표

무엇을 끝내고 싶은가

하네스 (감싸는 것)

맥락 · 경계 · 반복 · 검증 — 튜닝의 대상

모델 (core)

추론 엔진. 교체가 아니라 감싸서 일하게 만든다

## 영향력 순으로 잡는다

▶ 위에서부터 순서대로 — 맥락이 가장 중요

① 맥락을 준다

thin AGENTS.md + docs/  
가장 중요

② 경계를 정한다

config.toml:  
model · approval · sandbox

③ 반복을 패키징

Skill +  
린터·구조적 테스트

④ 검증을 강제

테스트 명령 =  
가장 영향력 큰 한 줄

## thin AGENTS.md + config.toml

▶ 루트에 thin AGENTS.md(~100줄), 깊은 지식은 docs/ · config.toml로 경계

```
# AGENTS.md (목차 골격, ~100줄)
## Setup/Tests  ## Do  ## Don't  ## When stuck
# 특정 파일은 @file, 저장소 밖은 MCP

# .codex/config.toml - 경계
model = "gpt-5.4"
approval_policy = "on-request" # 일상 개발
sandbox_mode = "workspace-write"
[profiles.review]
approval_policy = "never"; sandbox_mode = "read-only"
```

## 패키징하고, 강제한다

▶ 테스트 명령 한 줄이 하네스 전체에서 가장 영향력 크다

1

**반복 → Skill**

반복 워크플로를 SKILL.md로, \$이름으로 호출

2

**컨벤션은 기계로**

글 대신 린터·구조적 테스트로 강제

3

**변경 → 자동 테스트**

AGENTS.md에 테스트 명령 = 매 작업 후 자동 실행

4

**통과해야 완료**

파일 단위 빠른 검증(몇 초)으로 변경마다



## 하네스 없는 N vs 공유 하네스 위의 N

▶ 검증이 에이전트 사이의 가드레일이 된다

### 하네스 없는 N 에이전트

- 서로 다른 가정 → 드리프트
- 중복·범위 이탈이 기하급수
- 한 오류가 다음 입력을 오염 → 오류 전파
- 아무도 같은 기준이 없음

VS

### 공유 하네스 위의 N

- 공유 AGENTS.md(맥락)으로 정렬
- config(경계)로 같은 규칙
- 검증 = 에이전트 사이의 가드레일
- 오류가 다음으로 못 넘어감

SECTION 06

# 정리

## 오늘 3줄

- 멀티에이전트 = 분할(언제) + 패턴(순차·병렬·관리자·그래프) + 통신(메시지·핸드오프)
- 그 협업을 떠받치는 것이 하네스(맥락·경계·반복·검증)
- 에이전트가 여럿일수록 — 검증이 에이전트 사이의 가드레일

**더 똑똑한 모델보다, 잘 짜인 하네스 위의 잘 나뉜 팀이 멀리 간다.**

— 분할은 이득이 분명할 때만, 검증은 언제나